

Projeto para entrega

Parcelamento de compra

Praticando TDD com Vitest e TypeScript

RED escreva um teste que falha
GREEN faça o teste passar com o mínimo
REFACTOR melhore o código sem quebrar nada

ALUNO(A)

DATA DE ENTREGA

1 Contexto

Uma loja virtual precisa de uma função que calcule o parcelamento das compras. O cliente pode parcelar em até 18 vezes e, dependendo do número de parcelas escolhido, é aplicado um juro sobre o valor total da compra.

Seu trabalho é construir essa função usando **TDD (Test-Driven Development)**: primeiro você escreve o teste, vê ele falhar, e só depois escreve o código que faz ele passar.

2 Objetivos de aprendizagem

- Aplicar o ciclo **red, green, refactor** de forma disciplinada, um comportamento por vez.
- Transformar regras de negócio em casos de teste com o Vitest.
- Identificar e testar **casos de borda**, principalmente os limites entre faixas.
- Testar funções que lançam erro com `toThrow`.

3 Regras de juros

Parcelas	Juros sobre o valor total
1x a 4x	Sem juros
5x a 8x	5%
9x a 12x	8%
13x a 18x	10%

4 Assinatura da função

A função recebe o valor da compra e o número de parcelas, e retorna o valor de cada parcela e o total de parcelas.

```
export type ResultadoParcelamento = {
  valorParcela: number
  totalParcelas: number
}

export function calcularParcelamento(
  valorCompra: number,
  numeroParcelas: number
): ResultadoParcelamento {
  // sua implementação aqui
}
```

5 Regras de negócio

1. O juro é aplicado **uma única vez sobre o valor total** da compra. Só depois o valor é dividido pelo número de parcelas.
2. O valor da parcela deve ser **arredondado para 2 casas decimais**.
3. O número de parcelas deve ser um **inteiro entre 1 e 18**. Fora disso, a função lança um erro com a mensagem `Número de parcelas deve ser um inteiro entre 1 e 18`.
4. O valor da compra deve ser **maior que zero**. Caso contrário, a função lança um erro com a mensagem `Valor da compra deve ser maior que zero`.

6 Exemplos

Use a tabela abaixo como referência para escrever os seus testes. As linhas em destaque são os limites entre faixas, onde os bugs costumam aparecer.

Valor da compra	Parcelas	Total com juros	Valor da parcela
1000	1	1000	1000
1000	4	1000	250
1000	5	1050	210
1000	8	1050	131.25
1000	9	1080	120
1000	12	1080	90
1000	13	1100	84.62
1000	18	1100	61.11
100	3	100	33.33

Exemplos de erro

Chamada	Resultado esperado
<code>calcularParcelamento(1000, 0)</code>	Lança erro de número de parcelas
<code>calcularParcelamento(1000, 19)</code>	Lança erro de número de parcelas
<code>calcularParcelamento(1000, 2.5)</code>	Lança erro de número de parcelas
<code>calcularParcelamento(0, 3)</code>	Lança erro de valor da compra
<code>calcularParcelamento(-50, 3)</code>	Lança erro de valor da compra

7 Especificação dos testes

Seu arquivo de testes deve cobrir, no mínimo, os cenários abaixo. A ordem sugerida vai do caso mais simples para o mais trabalhoso, e é nessa ordem que recomendamos implementar.

```
import { describe, it } from 'vitest'

describe('calcularParcelamento', () => {
  describe('sem juros (1x a 4x)', () => {
    it('retorna o valor total em parcela única quando for 1x')
    it('divide o valor sem juros quando for 4x')
  })

  describe('com juros', () => {
    it('aplica 5% sobre o total quando for de 5x a 8x')
    it('aplica 8% sobre o total quando for de 9x a 12x')
    it('aplica 10% sobre o total quando for de 13x a 18x')
    it('aplica a faixa correta nos limites (4x, 5x, 8x, 9x, 12x, 13x)')
  })

  describe('arredondamento', () => {
    it('arredonda o valor da parcela para 2 casas decimais')
  })

  describe('validações', () => {
    it('lança erro quando o número de parcelas for menor que 1')
    it('lança erro quando o número de parcelas for maior que 18')
    it('lança erro quando o número de parcelas não for inteiro')
    it('lança erro quando o valor da compra for zero ou negativo')
  })
})
```

8 Como desenvolver

Siga o ciclo abaixo para **cada** cenário da especificação. Resista à vontade de escrever a função inteira de uma vez: o objetivo do exercício é o processo, não só o resultado.

1. RED

Escolha um único cenário, escreva o teste e rode. Ele precisa falhar, e pelo motivo certo.

2. GREEN

Escreva o mínimo de código para esse teste passar, sem quebrar os anteriores.

3. REFACTOR

Com tudo verde, melhore nomes, remova duplicação e organize as regras.

Faça commits pequenos

Faça pelo menos um commit a cada ciclo, com mensagens que indiquem a etapa, por exemplo `test: aplica 5% entre 5x e 8x` e `feat: implementa faixa de 5%`. O histórico de commits é a evidência de que você usou TDD.

Dica para os limites entre faixas

O teste dos limites é onde costuma aparecer o clássico bug de usar `<` no lugar de `<=`. Experimente usar `it.each` para testar vários limites com um único teste parametrizado.

9 Estrutura do projeto

Mantenha a estrutura abaixo. A função e o tipo ficam no mesmo arquivo, sem uso de classes.

```
parcelamento/  
├─ src/  
│   ├─ parcelamento.ts      # tipo + função  
│   └─ parcelamento.test.ts # testes com Vitest  
├─ package.json  
└─ tsconfig.json
```

Para configurar e rodar os testes:

```
# instalar as dependências  
npm install -D vitest typescript  
  
# rodar os testes em modo watch  
npx vitest
```

10 O que entregar

- ☐ Link do repositório público no GitHub com o projeto.
- ☐ Arquivo `parcelamento.ts` com a função e o tipo implementados.
- ☐ Arquivo `parcelamento.test.ts` cobrindo todos os cenários da especificação, com todos os testes passando.
- ☐ Histórico de commits mostrando os ciclos de TDD.
- ☐ `README.md` curto explicando como rodar o projeto e os testes.

11 Critérios de avaliação

Critério	O que será observado
Cobertura dos cenários	Todos os cenários da especificação têm teste implementado, incluindo validações e limites entre faixas.
Testes passando	A suíte roda sem erros com <code>npx vitest run</code> .
Regras de negócio	Juros, arredondamento e mensagens de erro seguem exatamente o que foi especificado.
Processo de TDD	Os commits mostram testes escritos antes da implementação, em ciclos pequenos.
Qualidade do código	Tipagem correta sem <code>any</code> , nomes claros e testes legíveis que funcionam como documentação.

12 Desafios extras

Opcionais, para quem terminar a parte principal:

1. Reescreva o teste dos limites usando `it.each` com a tabela de exemplos da seção 6.
2. Adicione o campo `valorTotal` ao retorno, com o total da compra já com juros, guiado por novos testes.
3. Repare que 3 parcelas de 33.33 somam 99.99. Proponha uma regra para que a soma das parcelas feche com o total (por exemplo, jogando a diferença na primeira parcela) e implemente com TDD.